WorkForce IQ

Government Workforce Productivity Platform

A portfolio case study

William Elturk
Senior Full-Stack Developer / Technical Lead
Solution Architecture · Data Warehousing · Power BI

C# / .NET 8 · Blazor Server · EF Core · SQL Server · SSIS · Power BI Report Server · ApexCharts · MinIO · OIDC SSO · Azure DevOps · xUnit / bUnit

The client is a government human-resources authority in the Gulf region. The client's identity, all environments and all data shown in this document are anonymized; screenshots are taken from a demo environment with synthetic data.

1 Context & Challenge

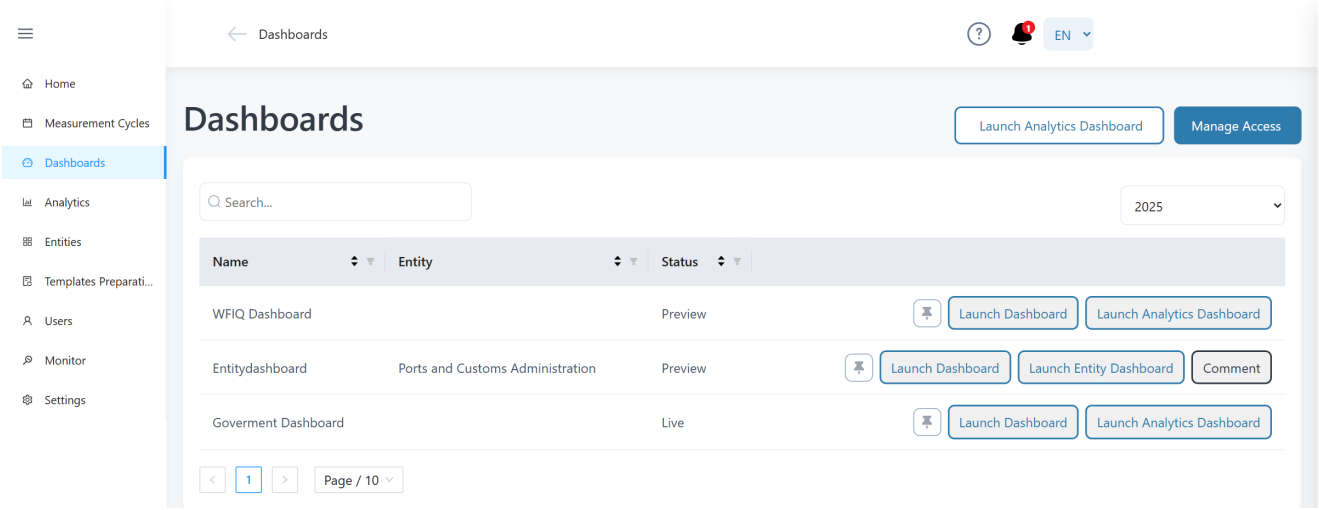
A Gulf-region government HR authority runs a national program that measures the productivity of the government workforce: every participating entity reports its services, financials and FTE allocations for each measurement cycle, and the program computes standardized productivity indices that leadership uses to compare entities, spot workforce inefficiencies (“workforce fat”) and steer policy.

When I joined, the process was heavily manual: Excel workbooks collected by email, ad-hoc validation, and reporting bottlenecked on a small analytics team. The program needed a secure, bilingual, self-service platform covering the full lifecycle — template preparation, guided data collection with quality gates, a governed calculation pipeline, and executive-grade reporting — for one government, roughly twenty entities and several hundred measured services.

1 national government program	**~20** government entities onboarded	**100s** of measured services	**2+ yrs** multi-phase agile delivery	**EN + AR** fully bilingual, complete RTL

What the platform does

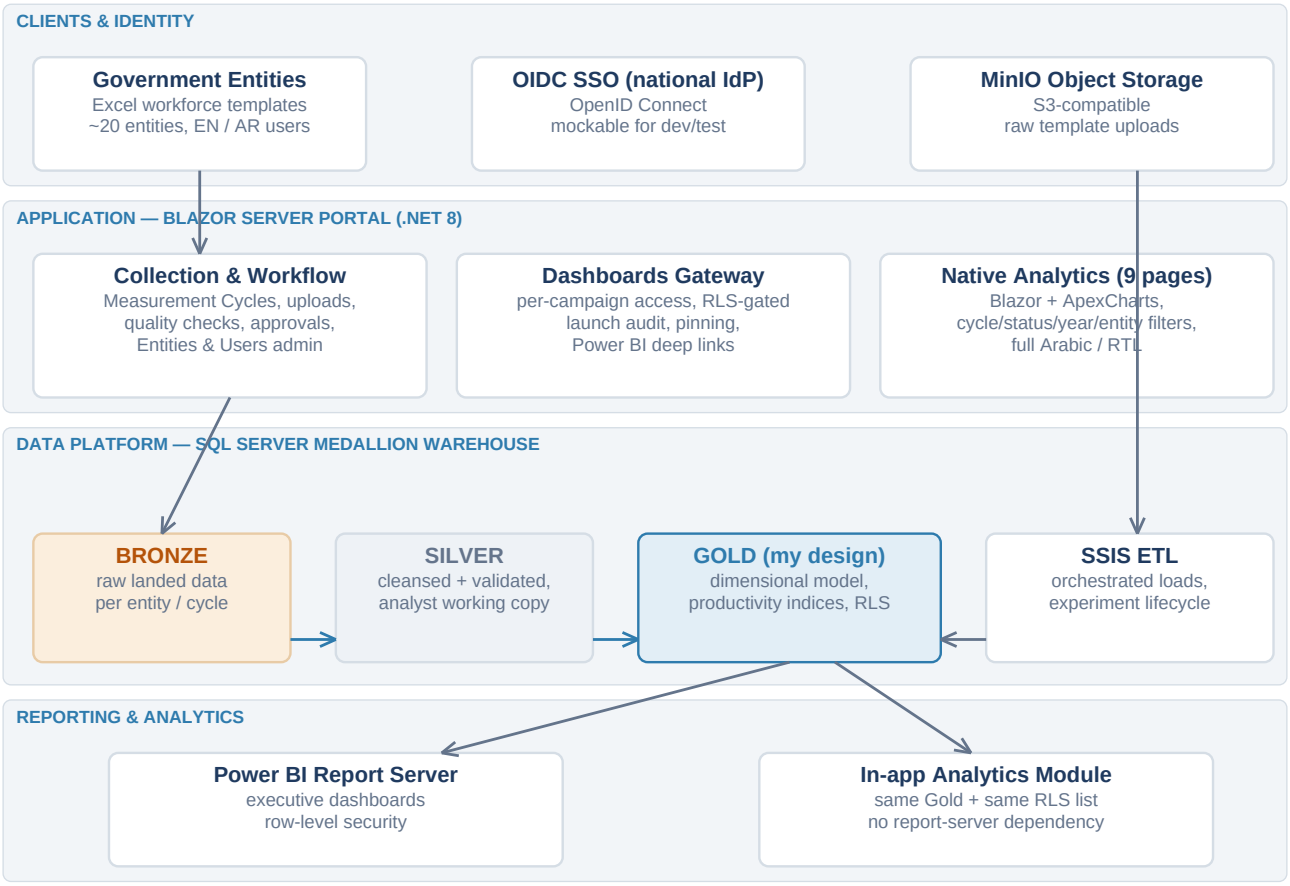
- **Template preparation** — generates per-entity Excel collection templates for every measurement cycle.
- **Guided collection** — entities upload workbooks through a workflow with automated data-type and business-rule quality checks, review steps and formal approvals.
- **Calculation pipeline** — SSIS ETL lands the data in a Bronze/Silver/Gold medallion warehouse where productivity indices are computed under an experiment lifecycle (Experiment → Preview → Live).
- **Reporting** — executive Power BI dashboards plus a native in-app analytics module, both reading the same Gold layer under the same row-level security.
- **Administration** — entity registry, user and role management, per-campaign dashboard access, audit logging and health monitoring.



Dashboards module — per-campaign Power BI gateway with launch audit and pinning (module I led)

2 Architecture

The platform is a Blazor Server application over a three-database SQL Server estate. The application database (EF Core, migration-managed) owns identity, campaigns, workflows and experiments. Uploaded workbooks land in MinIO object storage and are processed by SSIS into the medallion warehouse: raw **Bronze**, cleansed **Silver** (including the analyst working copy), and the reporting-grade **Gold** layer that I designed and own. Power BI Report Server and the native analytics module are deliberately thin consumers of the same Gold schema, sharing one row-level-security user list so a person sees identical data in both.



Solution architecture. Highlighted: the Gold layer (dimensional model, productivity-index calculations, RLS) — my sole-developer scope.

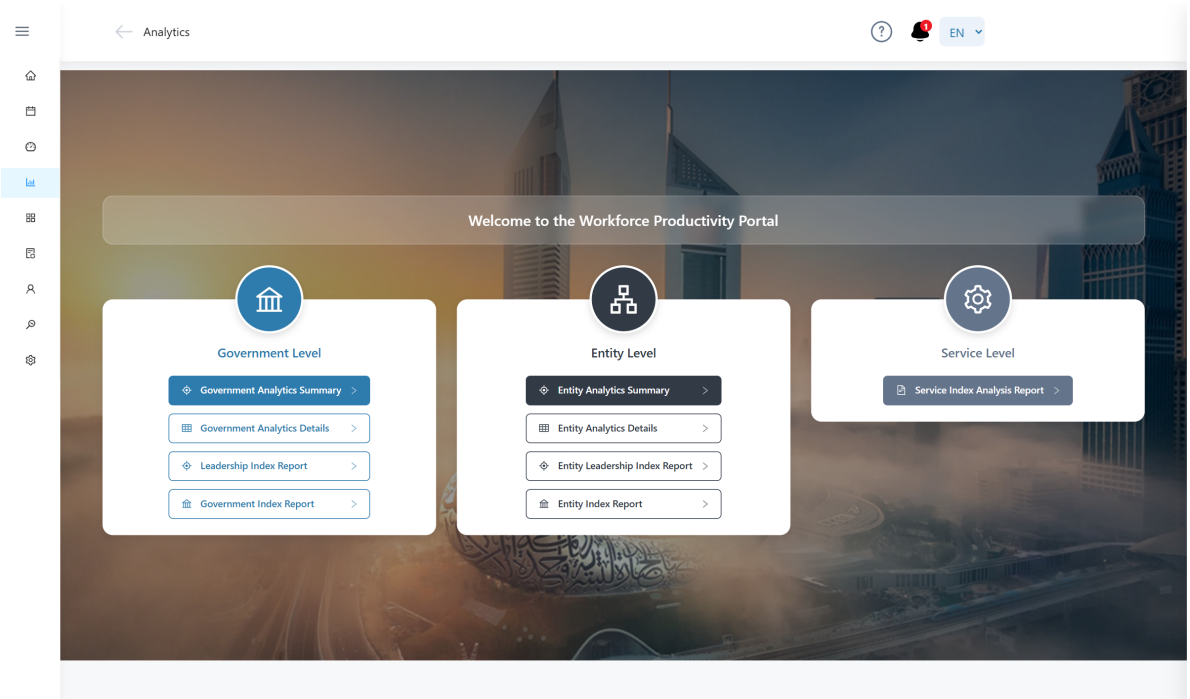
Architecture decisions I'm proud of

- **One Gold, many consumers.** Power BI and the in-app analytics never re-implement business logic; every measure lives once, in the warehouse. Migrating a report from Power BI to Blazor never changes a number.
- **Experiment lifecycle as data.** Calculation runs are versioned (Experiment / Preview / Live) inside the fact tables, so analysts can stage and compare results before publishing — no schema copies, no branch databases.
- **Permission-by-data.** Screen access, dashboard access and analytics scope are all table-driven and reconciled at startup; hiding a menu item is never the security boundary.

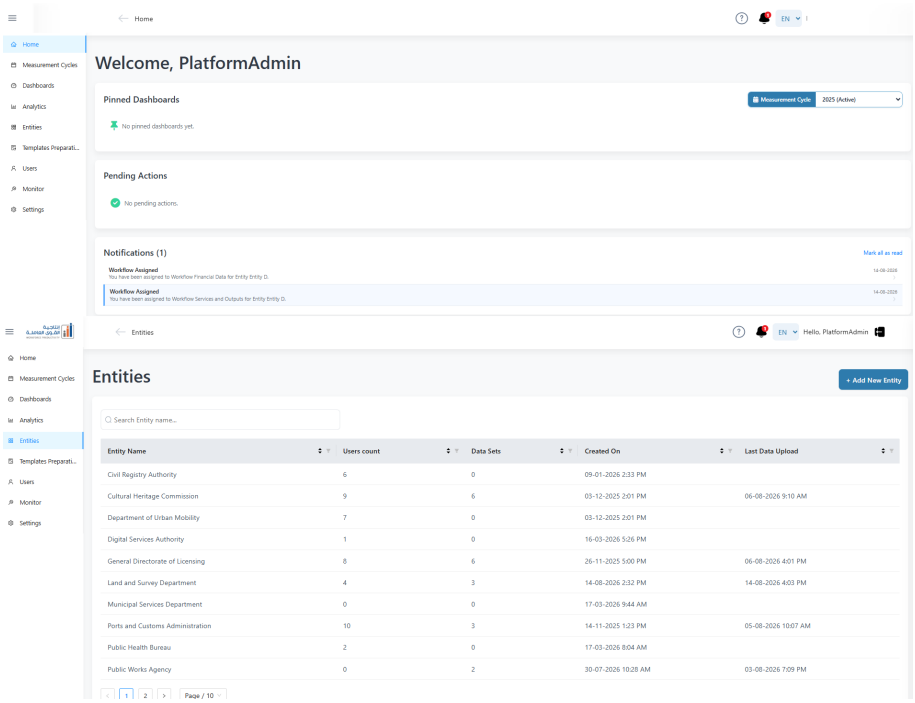
3 My Role — honestly tiered

On a multi-year program with an agile team, credit is shared. This is the honest split of my scope:

LED	**Dashboards module** — the Power BI gateway: per-campaign dashboard lists, warehouse-RLS-gated access, launch auditing, pinning, review comments. **Native Analytics module** — migrated 9 interactive Power BI report pages to Blazor + ApexCharts with shared cycle/status/year/entity filtering, deep-linkable URLs, and complete Arabic/RTL behavior. **Power BI dashboards** — authored and maintained the executive report suite across DEV/QA environments.
SOLE DEVELOPER	**Gold warehouse layer** — dimensional model, all productivity-index business logic and calculations, experiment versioning and the RLS design (dedicated section overleaf). **Users module** — invitations, roles, entity assignments. **Entities module** — the government-entity registry with bilingual names and logos.
CONTRIBUTOR	**Measurement Cycles module** — one of the developers on the collection workflows: upload steps, automated quality checks, model-output review and approval flows.



Native analytics landing — Government / Entity / Service report families (module I led)



Users & Entities administration (my sole-developer modules)

4 Deep Dive — The Gold Layer (data architecture)

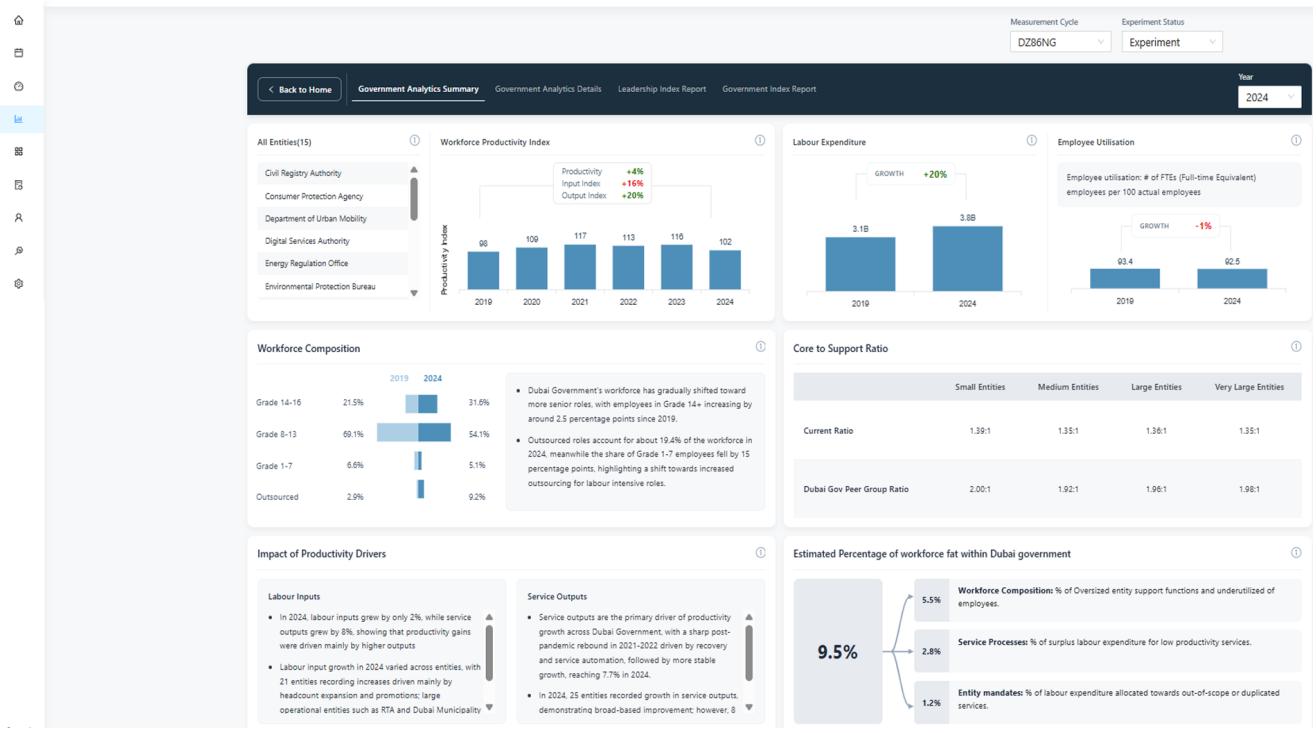
The Gold layer is the analytical contract of the whole platform: a star-schema warehouse whose grain, conformance and calculations every report depends on. I designed the model and own all of its business logic.

Model

- **Conformed dimensions** — entities (bilingual), services (per entity, per cycle, typed Individual/Collective/Support), and collection cycles keyed by measurement-cycle code so one warehouse hosts every cycle side by side.
- **Fact tables at three grains** — government-year, entity-year and service-year, plus labour-expenditure detail by grade and employment-distribution facts; every fact row carries its experiment lifecycle status.
- **Row-level security as data** — a warehouse user list maps portal users to a Power BI role (Government User vs Entity User) and an entity scope; both Power BI and the portal enforce from this one table.

Calculations

The core intellectual property is the productivity-index framework, which I implemented end to end: output indices built from service transaction volumes with quality adjustment, input indices from labour hours, salaries and grade mix, and **productivity = output / input** normalized to a base year (index = 100). On top of that sit growth decompositions (each service's contribution to entity growth, each entity's expenditure-weighted contribution to government growth) and the “workforce fat” estimation — the share of labour expenditure attributable to oversized support functions, underutilization, low-productivity services and out-of-scope mandates. Getting these right required translating economist-specified formulas into set-based SQL that reconciles exactly with the published Power BI measures.

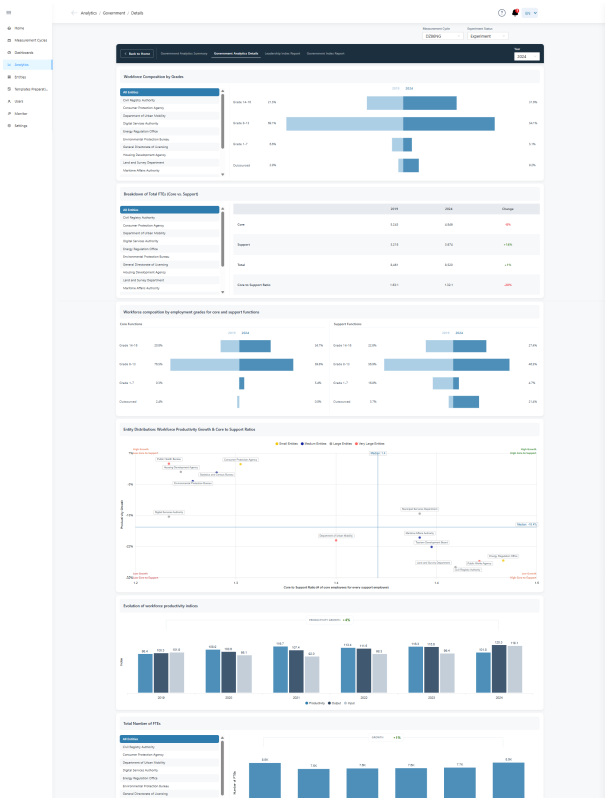


Government Analytics Summary — KPIs, index evolution and workforce composition, all computed in Gold

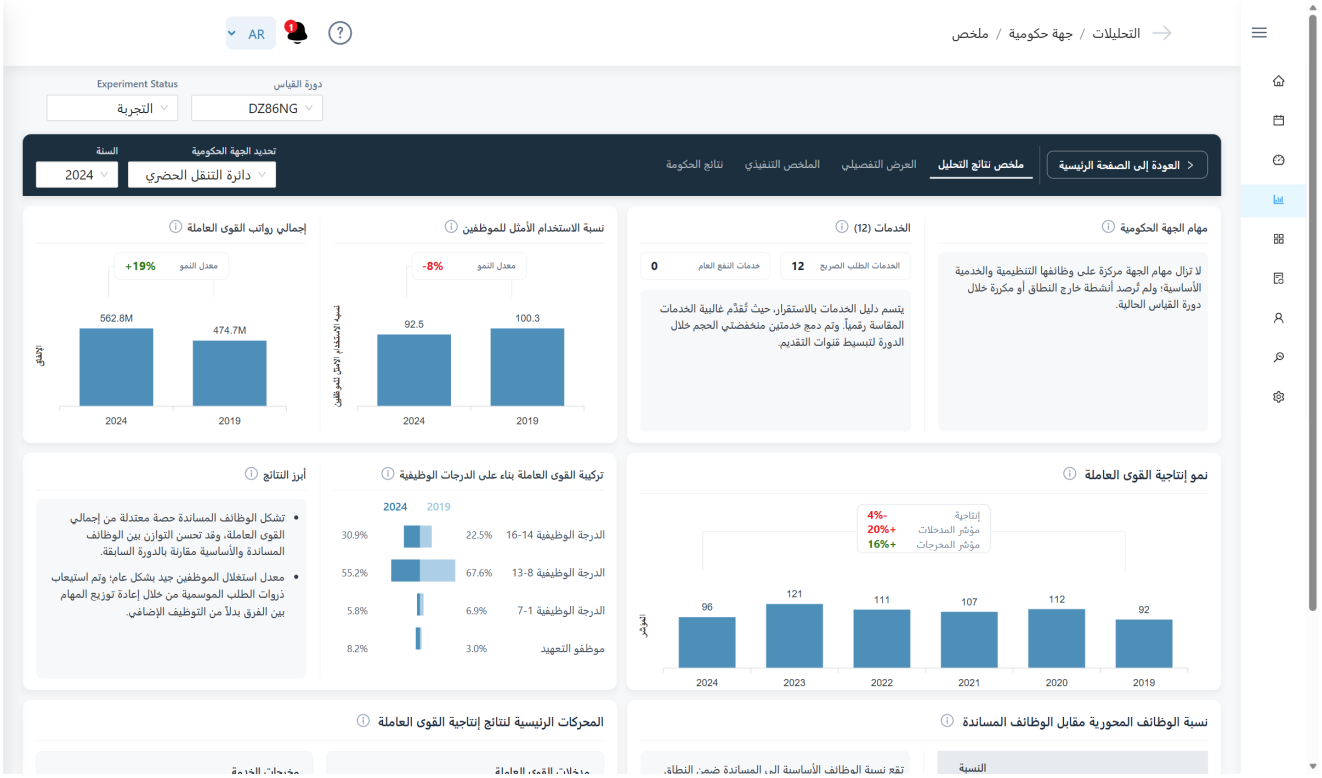
5 Deep Dive — Power BI → Blazor migration with full RTL

The program needed its nine most-used report pages inside the portal — same numbers, no report-server round-trip, and first-class Arabic. I led the migration to Blazor + ApexCharts: a shared filter system (measurement cycle, experiment status, year, entity) that synchronizes to the URL for deep-linking, reusable chart components (diverging in-table data bars, quadrant scatters with median annotations and collision-avoiding point labels, butterfly and stacked-column charts), and per-column proportional bar scaling matched to Power BI's data-bar behavior.

- **RTL was engineered, not toggled** — numbers pinned to Western digits with dot decimals in both languages (ICU's Arabic locale renders invisible bidi marks that scramble negative numbers), physical left/right kept for chart sign conventions while layout mirrors, and Arabic display names served from the warehouse.
- **Parity was measurable** — every migrated visual reconciles to its Power BI counterpart because both read identical Gold measures.



Entity Analytics Details — index evolution and service-contribution scatter (Blazor/ApexCharts port)



The same analytics in Arabic — mirrored layout, Western digits, correct bidi rendering

6 Deep Dive — Warehouse-driven row-level security

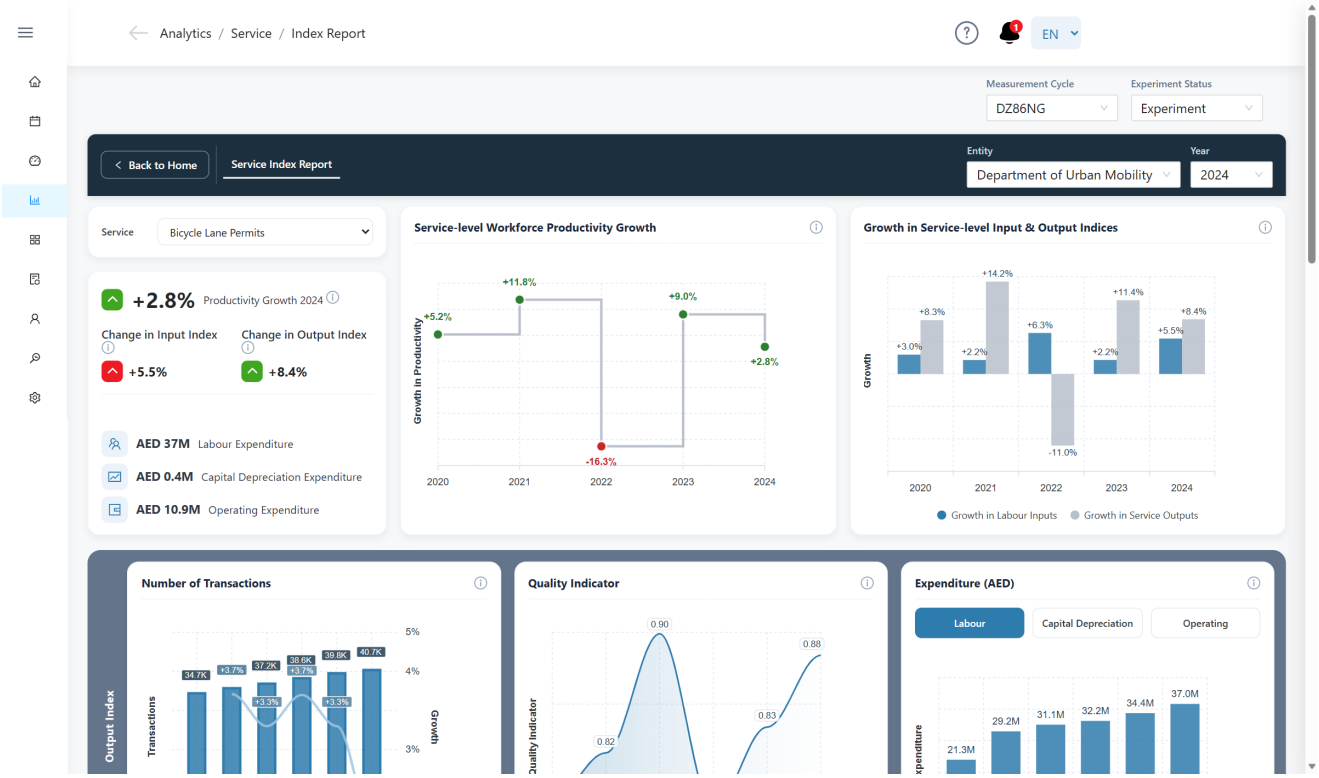
Access control had to be identical in two different reporting stacks. I made the warehouse RLS user list authoritative for both: Power BI applies it as classic RLS, and the portal resolves the same table into an analytics scope per user — a Government User sees everything, an Entity User is confined to their entity across every page, dropdown, dashboard tile and deep link. The scope is enforced in the data services (never just the UI): entity lists are filtered server-side, government-level routes are guarded, carried filter values from URLs or session state are re-validated against the user's scope before use, and the system fails closed when the warehouse is unreachable.

7 Deep Dive — Filters that survive language switches

Switching EN↔AR reloads the whole Blazor circuit, and analysts kept losing their cycle/status/year/entity selections mid-analysis. I designed a small persistence layer over protected browser session storage: selections are snapshotted on every filter change, restored once per circuit *before* default resolution runs (ordering was the subtle bug — a restored value read too late was silently overwritten), offered — never forced — back to the pages, and re-validated against RLS scope exactly like a deep-link value. Encrypted payloads survive key rotation by failing back to defaults.

8 Engineering practice

- **880+ automated tests** (xUnit for domain/services, bUnit for components) gate every pull request through an Azure DevOps pipeline; reviews run against a written repo standard (localization completeness, authorization on every route, display-only guarantees, no swallowed exceptions).
- **Test-guarded chart behavior** — even visual conventions (which side of zero a diverging bar grows, bidi-safe number formatting) are locked by unit tests, because in a bilingual government product those are correctness, not cosmetics.
- **Operability** — structured JSON logging, in-app audit log, health checks, and environment-gated features for safe incremental rollout.



Service Index Report — output/input/productivity matrices with transactions & quality trends

9 Outcomes

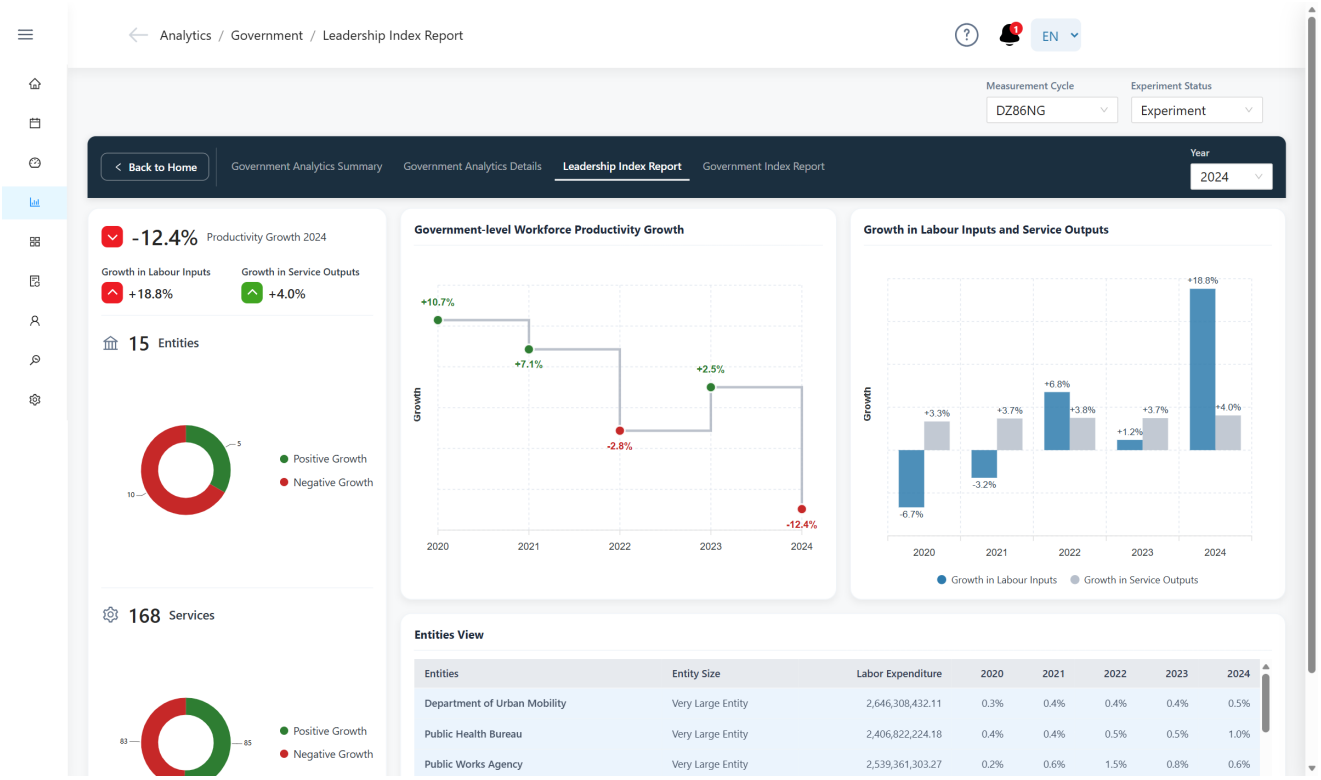
- Replaced a manual, email-driven collection process with a governed platform used across a national program: **~20 government entities**, hundreds of measured services, one auditable pipeline from Excel upload to executive dashboard.
- **Nine Power BI report pages** re-delivered natively in the portal with number-for-number parity, removing the report-server dependency for day-to-day analysis and unlocking full Arabic/RTL analytics.
- One warehouse, one security model: identical figures and identical access scopes across Power BI and the portal, with the calculation framework versioned through an experiment lifecycle instead of parallel databases.
- A codebase an agile team can move fast in: 880+ tests on every PR, table-driven permissions, and bilingual delivery as a default, not an afterthought.

What this project says about me

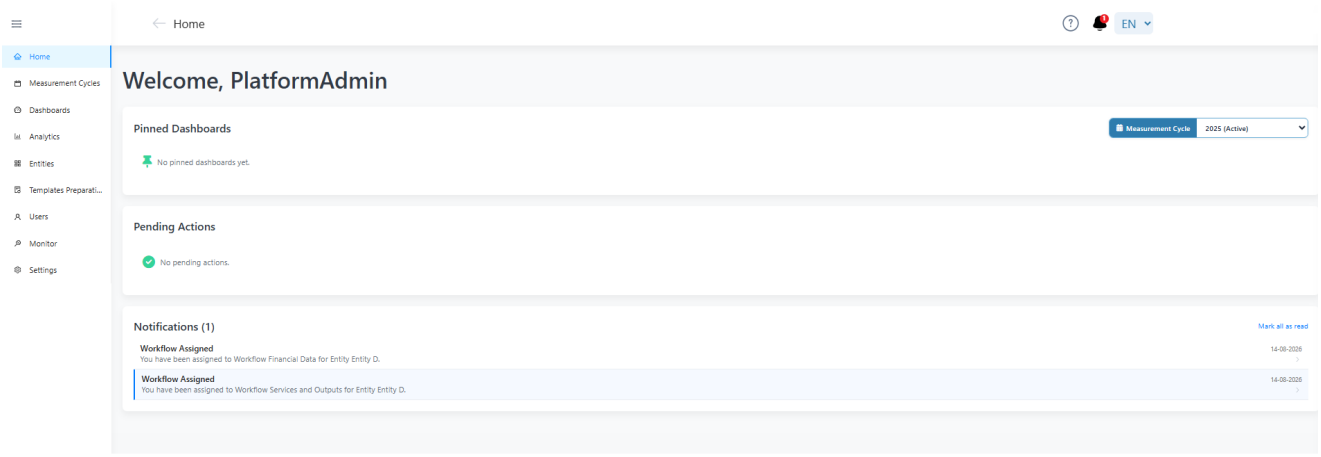
I operate across the full stack in the literal sense: I designed the warehouse layer economists trust, led the reporting experience executives read, and built the admin plumbing that keeps a multi-entity government program running — in two languages, under real security constraints, with the discipline (tests, reviews, honest attribution) that enterprise delivery demands.

William Elturk — Senior Full-Stack Developer / Technical Lead · Solution Architecture · Data Warehousing · Power BI

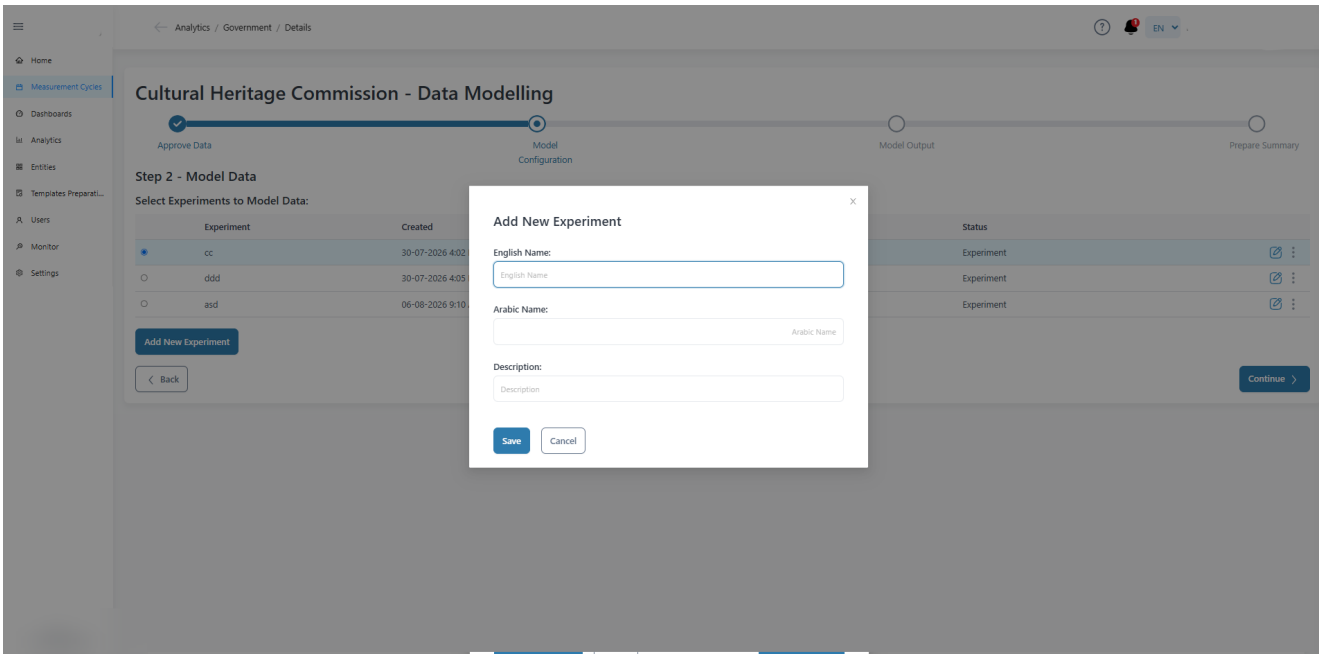
10 Gallery



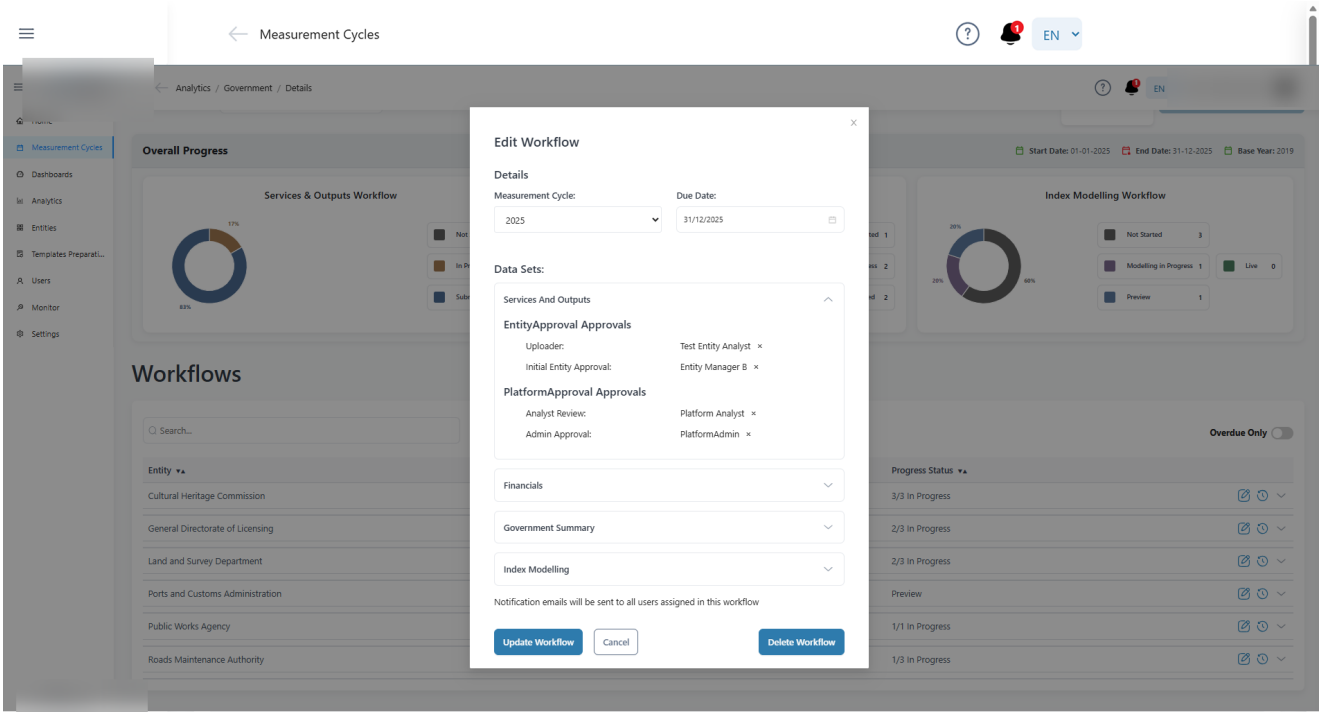
Leadership Index Report — government growth story, entity contributions and positive/negative splits



Home — pinned dashboards, pending actions and workflow notifications



Measurement Cycles — model configuration step with experiment management (contributor module)



Measurement Cycles — per-entity workflow orchestration with dataset approval chains (contributor module)